

Bericht DS02

Ziel des zweiten Übungsblattes war es, sich mit Schnappschussalgorithmen und konsistenten globalen Zuständen auseinanderzusetzen. Dazu wurde im Simulator sim4da ein verteiltes Bankhaus aus n Prozessen implementiert. Jeder Prozess verwaltet ein Konto mit einem Saldo, und die Prozesse überweisen sich gegenseitig zufällige Beträge. Bei einer Überweisung verringert der Sender sofort sein eigenes Konto, während der Empfänger seines erst erhöht, sobald die Nachricht ankommt. Da diese beiden Schritte nicht gleichzeitig passieren, ist zu fast jedem Zeitpunkt Geld unterwegs (Senderkonto verlassen, aber noch nicht beim Empfänger angekommen). Daher wird ein Schnappschussalgorithmus benötigt. Er hält zur Laufzeit einen konsistenten globalen Zustand des Systems fest, also alle Kontostände mit den Überweisungen, die aktuell unterwegs sind, ohne die Prozesse anzuhalten. Zunächst wurde in Aufgabe 1 das Grundgerüst implementiert. Anschließend wurde in Aufgabe 2 ein Schnappschussalgorithmus umgesetzt. In Aufgabe 3 wurde ein naiver Schnappschussalgorithmus implementiert, der ausschließlich die Kontostände erfasst und Überweisungsbeträge, die sich zum Zeitpunkt des Schnappschusses noch in den Kommunikationskanälen befinden, nicht berücksichtigt.

Aufgabe 01

Insgesamt gibt es n vernetzte Konten-Prozesse, und jedes Konto startet mit einem Saldo von 1000, sodass die Gesamtsumme $S = n \cdot 1000$ beträgt. Bei einer Überweisung wird der Betrag sofort vom Sender-Konto abgebucht, und der Empfänger erhält ihn, sobald die Nachricht bei ihm ankommt. Da `receive()` blockierend ist und kein Timeout besitzt, kann ein Prozess nicht gleichzeitig auf einen Sende-Zeitpunkt und eine eingehende Nachricht warten. Deswegen stößt jeder Prozess beim Start eine Überweisung an, und jede empfangene Überweisung löst eine neue aus. Sowohl der Betrag als auch das Ziel-Konto werden zufällig gewählt. Die Summe aller Kontostände plus alle gerade unterwegs befindlichen Überweisungen müssen immer S ergeben, es soll also weder Geld verloren gehen noch neues entstehen. Zusätzlich wurden zwei Vorgaben umgesetzt, um überhaupt inkonsistente Situationen erzeugen zu können. Zum einen wurde eine zufällige Latenz eingeführt, sodass Nachrichten länger unterwegs sind. Ohne diese Verzögerung wären die Kommunikationskanäle zum Zeitpunkt des Schnappschusses in den meisten Fällen leer. Zum anderen wurde die FIFO-Zustellung des Simulators deaktiviert. So verarbeitet ein Empfänger nicht mehr zwangsläufig die zuerst eingetroffene Nachricht, sondern wählt zufällig eine Nachricht aus seiner Warteschlange aus. Nachrichten können also umsortiert zugestellt werden. Aufgabe 1 bildet das Fundament und implementiert selbst noch keinen Schnappschussalgorithmus (`BankSimulation.java`).

Aufgabe 02

In Aufgabe 2 sollte ein konkreter Schnappschussalgorithmus umgesetzt werden. Die Wahl bestand zwischen dem Koordinator-/Einfärbeverfahren und dem Chandy-Lamport Verfahren. Ich habe mich für das Koordinator-/Einfärbeverfahren entschieden, bei dem ein Koordinator den Schnappschuss startet und die Knoten sich dabei einfärben. Jeder Prozess hat eine Farbe, wobei er vor dem Schnappschuss weiß ist und schwarz, nachdem er seinen Zustand festgehalten hat. Jede Überweisung trägt die Farbe ihres Absenders zum Sendezeitpunkt mit. Anhand dieser Farbe lässt sich entscheiden, ob eine Überweisung vor oder nach dem Schnitt losgeschickt wurde. Ziel ist ein konsistenter Schnitt, in dem kein Empfang ohne die zugehörige Sendung enthalten ist.

Der Koordinator löst den Schnappschuss aus, indem er per Multicast eine Nachricht an alle Prozesse schickt (SnapshotRequest). Wenn ein Prozess diese Nachricht empfängt, notiert er seinen aktuellen Kontostand, wird schwarz und meldet seinen aktuellen Zustand an den Koordinator zurück (StateReport).

Für den Kanalzustand gilt folgende Regel. Wenn ein schwarzer Prozess noch eine weiße Überweisung erhält, war sie zum Zeitpunkt des Schnitts unterwegs und er meldet sie deswegen als Kanalinhalt (ChannelReport) an den Koordinator. Der Koordinator darf deshalb nicht schon nach n Kontoständen aufhören, sondern muss abwarten, bis alle unterwegs befindlichen Nachrichten erfasst sind (nachrichtenbasierte Terminierung). Außerdem meldet jeder Prozess zusätzlich, wie viele weiße Überweisungen er verschickt und empfangen hat. Aus der Differenz kann der Koordinator also berechnen, wie viele Kanalmeldungen noch fehlen. Das ist sowieso notwendig, weil receive() kein Timeout besitzt und der Koordinator die exakte Anzahl kennen muss.

Ein weiterer Fall ist, dass ein weißer Prozess eine schwarze Überweisung erhält, also eine Nachricht aus der Zukunft. Würde er den Betrag einfach gutschreiben, zählt er doppelt. Die Vorlesung nennt hierfür zwei Lösungsvarianten. Entweder die Nachricht verzögern und zwischenspeichern, bis der Prozess seinen eigenen Schnappschuss macht, oder die Nachricht als Auslöser nehmen und sofort selbst fotografieren. Die zweite Variante ist robust, benötigt keinen Zwischenspeicher und breitet den Schnappschuss schneller aus, weswegen diese Variante implementiert wurde.

Als Annahmen des Verfahrens gelten wie in der Vorlesung keine Nachrichtenverluste und keine Prozessabstürze. Die Annahme keine Nachrichtenverluste ist besonders wichtig, weil der Koordinator über die Differenz $\text{Summe}(\text{whiteSent}) - \text{Summe}(\text{whiteRecv})$ genau abzählt, wie viele Kanalmeldungen noch eintreffen müssen. Würde eine Überweisung verloren gehen, würde diese Rechnung nicht aufgehen und der Koordinator würde ewig auf eine fehlende Meldung warten. Anders als beim Chandy-Lamport-Verfahren wirkt sich die Abschaltung der FIFO-Zustellung bei dem Koordinator-/Einfärbefahren Verfahren nicht auf die Korrektheit aus. Jede Basisnachricht trägt die Farbe ihres Absenders mit. Ob eine Überweisung zum Kanalzustand gehört, entscheidet also allein die Farbe und nicht die Ankunftsreihenfolge. Auch die Terminierung bleibt korrekt, weil Nachrichten gezählt werden und die Reihenfolge irrelevant ist.

Am Ende summiert der Koordinator alle Kontostände und Kanalinhalt und gibt den vollständigen globalen Zustand aus.

Aufgabe 03

In Aufgabe 3 sollte gezeigt werden, dass der Schnappschuss tatsächlich einen konsistenten Zustand liefert. Er soll während des laufenden Betriebs ausgelöst werden, also die Prozesse überweisen weiter, während der Koordinator fotografiert. Um zu zeigen, dass die erfasste Gesamtsumme (Konten + Kanäle) immer S ergibt, wurde der Algorithmus mehrfach und mit verschiedenen n ausgeführt. In jedem Lauf ergab sich eine andere Verteilung auf Konten und Kanäle, die Summe war aber immer gleich S . Dass der Schnappschuss tatsächlich während des laufenden Betriebs ausgelöst wurde, zeigt sich unter anderem darin, dass ein großer Teil des Geldes zum Schnitt-Zeitpunkt unterwegs war und dementsprechend den Kanälen erfasst wurde.

n	Summe Kontostände	Summe unterwegs	Gesamt	Soll
3	1648	1352	3000	3000
5	3429	1571	5000	5000
25	12383	12617	25000	25000
50	25065	24935	50000	50000
100	47225	52775	100000	100000

Zum Vergleich wurde ein naiver Schnappschuss implementiert (BankSnapshotNaive). Dort fragt der Koordinator nur die Kontostände ab und ignoriert die Kanäle. Das Ergebnis ist ungleich dem Soll und es verschwinden Beträge. Das Geld, das zum Schnitt-Zeitpunkt unterwegs war, steckt in keinem Kontostand und wird auch nicht erfasst.

n	Summe Kontostände	Summe unterwegs	Gesamt	Soll
3	583	-	583	3000
5	2029	-	2029	5000
25	12907	-	12907	25000
50	27337	-	27337	50000
100	46826	-	46826	100000

Der Begriff des konsistenten Schnitts ($e_2 \in S \wedge e_1 <_k e_2 \Rightarrow e_1 \in S$) ist folgender. Eine Überweisung besteht aus einem Senden Ereignis (e_1 : Abbuchen) und dem Empfangen (e_2 : Gutschreiben). Ein konsistenter Schnitt darf keinen Empfang erhalten, wenn die zugehörige Sendung fehlt. Der naive Schnappschuss fragt die Kontostände zu unterschiedlichen Zeitpunkten ab und kann dadurch einen inkonsistenten Schnitt erzeugen. Entweder wird e_1 erfasst, e_2 aber nicht, Beträge, die unterwegs sind, bleiben unberücksichtigt (Geld verschwindet), oder e_2 wird erfasst, ohne dass e_1 im Schnitt liegt, also eine Nachricht aus der Zukunft, bei der Geld doppelt zählt (Geld entsteht).

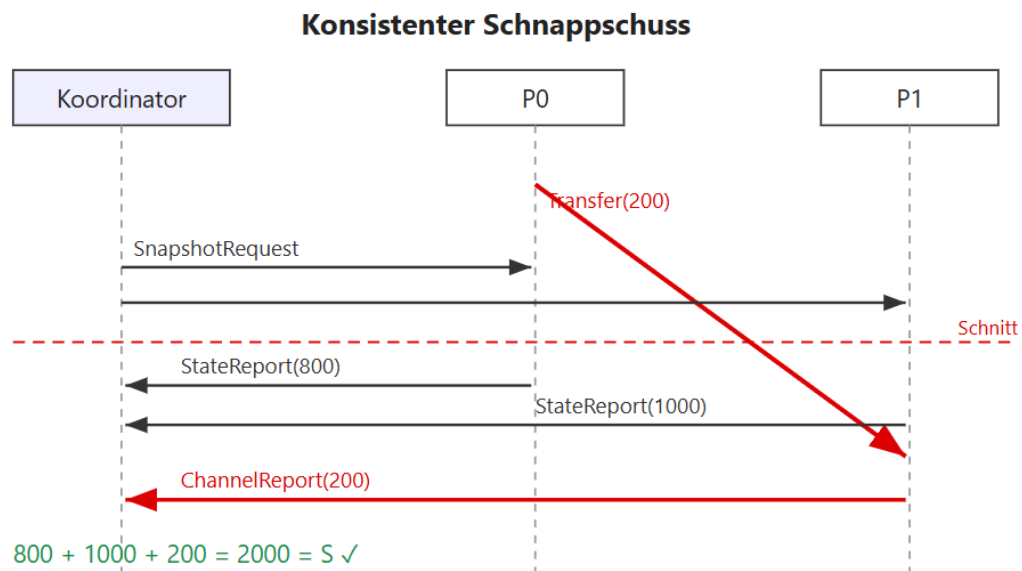
Zuletzt wurde gemessen, wie teuer ein einzelner Schnappschuss ist, also wie viele Kontrollnachrichten er kostet. Kontrollnachrichten sind alle Nachrichten, die nur für den Schnappschuss verschickt werden. Die Anfrage des Koordinators (SnapshotRequest), die Zustandsmeldungen der Konten (StateReport) und die Kanalmeldungen (ChannelReport). Die Überweisungen selbst (Transfer) zählen nicht dazu. Pro Schnappschuss schickt der Koordinator n Anfragen (eine an jedes Konto), bekommt n Kontostände zurück und dazu für jede unterwegs befindliche Überweisung eine Kanalmeldung. Die Anzahl der Kontrollnachrichten wächst also mit n. Bei n=3 ergeben sich 9 Kontrollnachrichten. Bei n=5 sind es 15 und bei n=100 sind es 300. Es lässt sich ableiten, dass sich 3n Kontrollnachrichten und somit lineares Wachstum ergibt. Pro Schnappschuss fallen also $O(n)$ Kontrollnachrichten an, die nichts zur eigentlichen Anwendung beitragen. Bei großem n oder häufigem Auslösen belastet dieser Overhead das System. Zusätzlich wurde eine weitere Größe erfasst. Der Anteil des unterwegs befindlichen Geldes (Summe unterwegs / S) lag je nach Lauf bei etwa 48-65%. Dieser Anteil erklärt gleichzeitig das Versagen des naiven Schnappschusses und er belegt, dass zum Schnitt-Zeitpunkt tatsächlich weiter überwiesen wurde.

Ein weiterer Nachteil, der in der Vorlesung benannt wurde, war, dass der Algorithmus nicht verteilt genug ist. Der Koordinator löst den Schnappschuss aus, kennt n und sammelt alle Meldungen ein. Damit laufen n Zustandsmeldungen und alle Kanalmeldungen auf einem einzigen Knoten zusammen. Das widerspricht dem Grundgedanken eines verteilten Systems, in dem es idealerweise, keinen Single Point of Failure gibt. Außerdem ist der Koordinator ein Flaschenhals, weil er alle Antworten

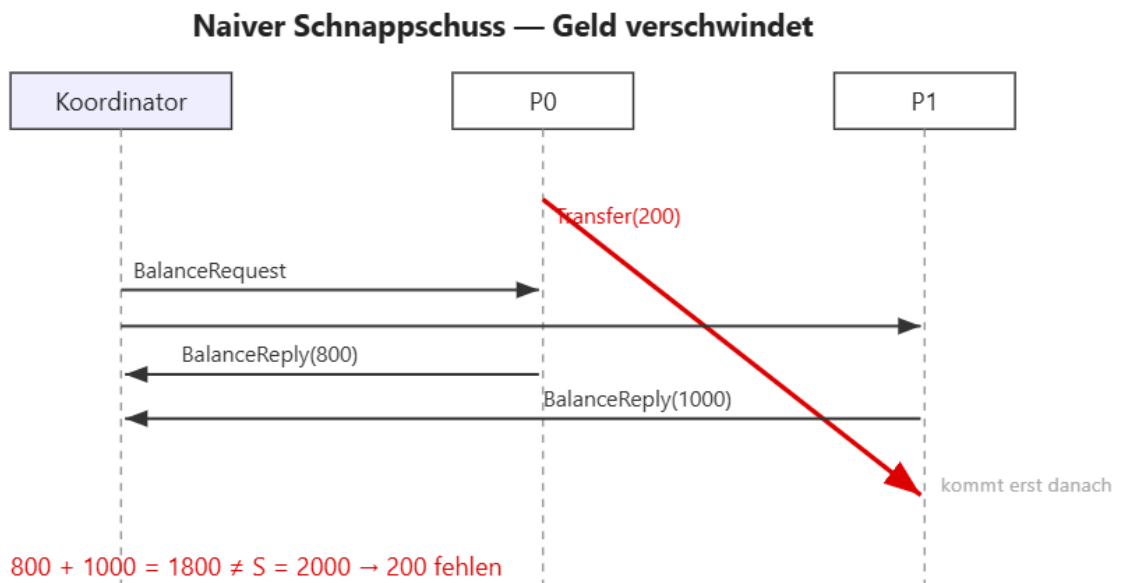
verarbeiten muss. Der Schnappschuss ist auch erst fertig, wenn auch der langsamste Prozess geantwortet hat.

Die folgenden Sequenzdiagramme veranschaulichen den Unterschied am selben Szenario:

- zwei Konten
- $S = 2000$
- P0 überweist 200 an P1
- zum Schnitt-Zeitpunkt unterwegs



Beide Konten melden ihren Stand an den Koordinator und die Überweisung von 200, die aktuell unterwegs ist, wird zusätzlich als ChannelReport erfasst. So ergibt sich als Gesamtsumme $800 + 1000 + 200 = 2000 = S$.



Beim Naiven Schnappschuss fragt der Koordinator die Kontostände ab, die Überweisung, die unterwegs ist, wird jedoch nicht erfasst. So ergibt die Summe $800 + 1000 = 1800 \neq S$. Der Betrag von 200 ist in diesem Fall verschwunden.

Fazit

Insgesamt zeigt das Übungsblatt, dass sich ein konsistenter globaler Zustand in einem verteilten System auch ohne globale Uhr zuverlässig erfassen lässt, zumindest, wenn man neben den lokalen Zuständen auch die unterwegs befindlichen Nachrichten berücksichtigt. Der Vergleich mit dem naiven Schnappschuss macht deutlich, dass die Kanalzustände den entscheidenden Unterschied zwischen einem korrekten und einem widersprüchlichen Ergebnis ausmachen. Gleichzeitig wurde deutlich, dass beim Koordinator-/Einfärbeverfahren ein zentraler Koordinator (SPOF) erforderlich ist und auch steigt mit steigender Prozessanzahl das Nachrichtenaufkommen.